Illoominate – Data Shapley Values for Debugging Neighborhood-based Recommender Systems*

BARRIE KERSBERGEN, Bol & University of Amsterdam, Amsterdam, The Netherlands
OLIVIER SPRANGERS, Nixtla, Amsterdam, The Netherlands
BOJAN KARLAŠ, Harvard University, Boston, United States
MAARTEN DE RIJKE, University of Amsterdam, Amsterdam, The Netherlands
SEBASTIAN SCHELTER, BIFOLD & TU Berlin, Berlin, Germany

Machine learning-powered recommender systems help users find items they like. Issues in the interaction data processed by these systems frequently lead to problems, e.g., to the accidental recommendation of low-quality products or dangerous items. Such data issues are hard to anticipate upfront, and are typically detected post-deployment after they have already impacted the user experience. We argue that a principled data debugging process is required during which human experts identify potentially hurtful data issues and preemptively mitigate them. Recent notions of "data importance," such as the Data Shapley value, represent a promising direction to identify training data points likely to cause issues. However, the scale of real-world interaction datasets makes it infeasible to apply existing techniques to compute the Data Shapley value in recommendation scenarios.

We tackle this problem by introducing the KMC-Shapley algorithm for the scalable estimation of Data Shapley values in neighborhood-based recommendation on sparse interaction data. We conduct an experimental evaluation of the efficiency and scalability of our algorithm on both public and proprietary datasets with millions of interactions, and showcase that Data Shapley value identify impactful data points for two recommendation tasks in e-commerce. Furthermore, we discuss applications of Data Shapley values on real-world click and purchase data in e-commerce, such as identifying dangerous products or improving the ecological sustainability of product recommendations. We implement our approach in the Illoominate library and release it under an open license.

CCS Concepts: • **Information systems** → Recommender systems.

Additional Key Words and Phrases: Data importance, Data debugging, Data Shapley value, Neighborhood-based recommendation

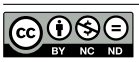
ACM Reference Format:
Barrie Kersbergen, Olivier Sprangers, Bojan Karlaš, Maarten de Rijke, and Sebastian Schelter. 2026. Illoominate – Data Shapley Values for Debugging Neighborhood-based Recommender Systems. *ACM Trans. Recomm. Syst.* 1, 1 (January 2026), 27 pages. https://doi.org/10.1145/3828670

1 Introduction

Recommender systems powered by machine learning help users to find the items that they need from large inventories. Real-world recommenders operate on large datasets that capture complex

*This is an extended version of [23], which was published at the ACM Conference on Recommender Systems 2025.

Authors' Contact Information: Barrie Kersbergen, Bol & University of Amsterdam, Amsterdam, The Netherlands, bkersbergen@bol.com; Olivier Sprangers, Nixtla, Amsterdam, The Netherlands, olivier@nixtla.io; Bojan Karlaš, Harvard University, Boston, United States, bkarlas@mgh.harvard.edu; Maarten de Rijke, University of Amsterdam, Amsterdam, The Netherlands, m.derijke@uva.nl; Sebastian Schelter, BIFOLD & TU Berlin, Berlin, Germany, schelter@tu-berlin.de.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2770-6699/2026/1-ART
https://doi.org/10.1145/3828670

interactions between users and items. However, issues in this interaction data frequently lead to system failures. Examples from the e-commerce domain include the accidental recommendation of dangerous items [37, 50], externally manipulated rankings [5], or third-party products with low-quality metadata [38, 49]. Furthermore, the data collection process is exposed to various sources of noise, e.g., "multi-journeys," where a single user shops for multiple things simultaneously (such as presents for family members of different age groups), or "unnatural interaction patterns" caused by bots crawling the website to record prices.

To make matters worse, these issues are hard to anticipate upfront, and are typically detected post-deployment after they have already impacted the user experience. For example, Bol, one of Europe's largest e-commerce platforms, features 51 million products offered by more than 45,500 external partners to 26 million active users.[1] At Bol, we recently became aware of a situation where sensitive adult items were included in the recommendations on product pages of children's toys, due to the unexpected co-occurrence of these types of products in some historical browsing sessions. This incident prompted an urgent live patch of the system with custom filtering rules, followed by the manual identification and removal of the session data in which these unwanted co-occurrences appeared.

1.1 Data Debugging via Data Importance

Preventing such predicaments requires a *principled data debugging* process during which human experts identify potentially hurtful data issues and preemptively mitigate them. However, the scale of real-world training datasets renders this process prohibitively expensive and time-consuming.

This shift reflects a growing awareness of the critical role of training data in shaping model behavior. A promising approach is to identify data points likely to cause issues via *data importance*, a concept recently introduced in the machine learning community. A popular notion of importance is the *Data Shapley value* (DSV) [12], which has been recognized for its effectiveness in some data debugging tasks [20, 21], including model auditing, dataset pruning, and outlier detection.

However, existing research on the DSV primarily focuses on classification tasks and has not been applied to recommendation tasks (except for a concurrent work on data pruning [58]). Moreover, current methods to overcome the exponential complexity of calculating the Data Shapley value are not suitable for large-scale recommender models (Section 3). Some techniques treat the model as a black box and incur repeated retraining [12, 27], which is infeasible for large datasets, while others make assumptions about the additivity of model quality metrics [19, 21] that do not apply to the ranking metrics of recommender systems.

1.2 Overview and Contributions

Our work aims to bridge this gap. We focus on the large class of neighborhood-based recommendation models on sparse interaction data [2, 6, 9, 17, 18, 28, 29, 31, 33, 35, 41, 42]. We detail how to use the characteristics of such KNN models to efficiently compute Data Shapley values for datasets with millions of interactions (Section 4.3). Based on these characteristics, we design a specialized variant of a recent Monte Carlo-based algorithm [12] for estimating the DSV, which can skip certain expensive computations in many cases (Section 4.4). In particular, we provide the following contributions:

- *KMC-Shapley algorithm.* We design the KMC-Shapley algorithm for the estimation of the Data Shapley value for neighborhood-based recommendation on sparse interaction data. This algorithm is a scalable variant of a recent Monte Carlo-based algorithm [12] for estimating the DSV, tailored to KNN models (Section 4).

[1] https://over.bol.com/en/about-bol/#facts-and-figures, accessed in December 2025.

- *Experimental evaluation.* We conduct an experimental evaluation of the efficiency and scalability of our algorithm on both public and proprietary datasets with millions of interactions. Moreover, we showcase that the DSV identifies impactful data points for two recommendation tasks in e-commerce (Section 5).
- *Discussion of applications in e-commerce.* We discuss applications of the DSV on click and purchase data in e-commerce from Bol, such as identifying dangerous and low-quality products as well as improving of the ecological sustainability of product recommendations via data pruning (Section 6).
- *Exploratory analysis of transferability of DSVs.* We explore how well our KNN-derived Data Shapley values can serve as proxies for data importance in neural recommenders (Section 7).
- *Open source implementation.* We implement our approach in the Illoominate library and make it available at https://github.com/bkersbergen/illoominate.

2 Related Work

To the best of our knowledge, we are the first to explore data importance for KNN-based recommenders. The only other application of the DSV in recommender systems (that we are aware of) is the selection of data points for pruning [58], where it successfully uncovers errors introduced by synthetic random noise.

2.1 KNN-based Recommender Systems

KNN models have a long tradition in recommender systems, ranging from classical collaborative filtering [31, 41, 42] to recent state-of-the-art algorithms in session-based [6, 18, 33, 35], session-aware [28], next-basket [9, 17, 29], and within-basket recommendation [2], to applications for conversational recommendation [56]. Our work uses the characteristics of these models, and we experiment with existing state-of-the-art KNN models for session-based recommendation and next-basket recommendation. As detailed for some exemplary cases in Section 4.2, most KNN algorithms are easy to map to our computational model, which automatically makes them compatible with KMC-Shapley, and enables the computation of data importance scores for their training data.

2.2 Error Detection for ML Data

Detecting errors in data is a core research direction in data management [1, 30], and several approaches tailored to ML applications have been proposed. Google TFX [4] and Deequ [44, 45] infer integrity constraints for ML data based on data profiling, and follow-up approaches learn to validate ML data from historically observed data partitions [40, 46], few-shot examples [14] or via self-supervised learning [32]. Our work shares the focus on scalability and efficiency with these approaches, which is a prerequisite for the applicability to real-world data. In contrast to many existing approaches in this area, our approach does not rely on hand-crafted rules, heuristics applied to data profiling results, or encoded domain knowledge. As demonstrated in Section 5, DSVs identify erroneous data items automatically via their negative impact on the predictive performance of a recommender system.

2.3 Data Importance

Determining the importance of a data point for an ML model is a core question in data-centric AI [13], with many applications in data debugging and data selection. Prominent notions of data importance include the Data Shapley value [12], and generalisations such as the Beta Shapley value [27] and the Data Banzhaf value [54]. The efficient estimation of such data importance scores is an active area of research with promising results for special classes of models such as KNN

classifiers [19], which can be used as a proxy for more expensive models. Datascope [21] and ArgusEyes [43] use Data Shapley values to debug the outputs of ML pipelines, while Gopher [39] and Rain [10, 55] use estimates of the leave-one-out error computed via influence functions [25] for data debugging. Besides ML-specific scenarios, there is a wide variety of applications of the Shapley value in data management; see [36]. Our work builds on established notions of data importance, and can be seen as another instantiation of the idea of specializing the importance computation to particular model families. As stated before, we are the first to explore data importance for recommendation models and data.

2.4 Relationship to Previous Work

This journal paper extends a recent paper of ours published at ACM RecSys 2025 [23] with novel content along several dimensions. We add several experiments and applications, such as an evaluation for next-basket recommendation models (Section 5.2.2), an empirical analysis of the relationship between data importance and session characteristics for click data in a real-world recommender system (Section 6.2), and a novel application to ecological sustainability (Section 6.3). To motivate follow-up work, we also include a novel exploratory experimental study on the transferability of KMC-Shapley values to neural session-based recommendation models (Section 7). Furthermore, we provide additional details on core parts of the paper, e.g., examples of how recent KNN algorithms map to our abstract computational model (Section 4.2), the exact algorithm to efficiently compute leave-one-out errors for KNN models on sparse data, and a brief overview of our open source implementation.

3 Problem Statement

We formally introduce data importance (see Figure 1, inspired by [12]) and the scalability problem at the focus of this paper.

3.1 Data Importance

The goal of data importance is to quantify the impact of each individual training data point on the quality of a machine learning model [13]. Many notions of importance rely on measuring the impact of excluding a given data point from the model's training data (or certain subsets of it) [12, 27, 54]. Let $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ denote the set of n training data points whose importance we want to compute. Let the *utility function* $V(S)$ measure the performance of a model trained on a subset $S \subseteq \mathcal{D}$ of the training data. For example, V might compute the recall of a recommendation model on held-out data $\mathcal{D}_{val}$. We discuss the two most prominent notions of data importance here.

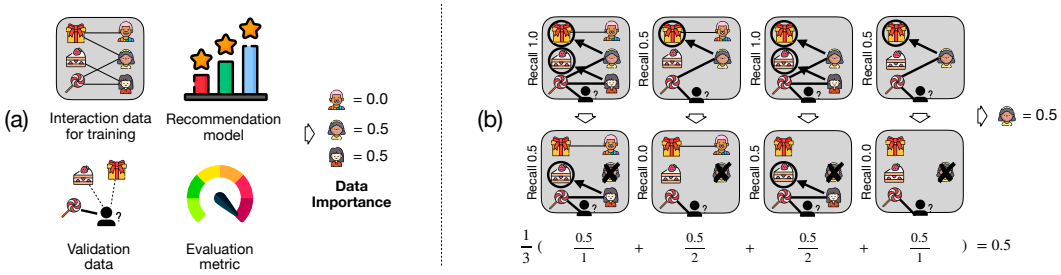


Fig. 1. (a) High-level overview of data importance for recommender systems. (b) The importance of each piece of data (e.g., the interactions of a given user) is a weighted average of its contribution to each subset of the remaining interactions. The importance scores are determined by Equation (2).

3.1.1 Leave-One-Out error (LOO). The simplest way to measure the importance of a data point $\mathbf{x}_i \in \mathcal{D}$ is its leave-one-out error, i.e., the change in utility V when the data point $\mathbf{x}_i$ is excluded from the training data $\mathcal{D}$:

$$V(\mathcal{D}) - V(\mathcal{D} \setminus \{\mathbf{x}_i\}). \tag{1}$$

While the LOO is easy to compute, it is empirically found to be highly noisy [27] and typically outperformed by more complex notions of importance in its helpfulness for downstream tasks [13].

3.1.2 Data Shapley value (DSV). Recently, the Data Shapley value (DSV) has been proposed as an equitable way to measure the importance of training data points for a machine learning model [12]. The DSV determines the "value" ϕ_i of each data point $\mathbf{x}_i$ from the data $\mathcal{D}$ based on the well-known Shapley value from game theory [26]:

$$\phi_i = \frac{1}{n} \sum_{S \subseteq \mathcal{D} \setminus \{\mathbf{x}_i\}} \binom{n-1}{|S|}^{-1} V(S \cup \{\mathbf{x}_i\}) - V(S). \tag{2}$$

The DSV measures the weighted average of the marginal contribution $V(S \cup \{\mathbf{x}_i\}) - V(S)$ of adding the data point $\mathbf{x}_i$ to all $2^{|\mathcal{D}|-1}$ subsets S of the training data $\mathcal{D}$. See Figure 1(b) for a toy example. The DSV is challenging to compute but has been shown to work well empirically, e.g., for tasks like detecting mislabeled data [12, 21, 27, 43]. Note that a major advantage of data importance techniques for our scenario is that they are able to identify various kinds of erroneous data points without requiring explicit knowledge of the actual error types upfront, in contrast to classical supervised learning approaches.

3.2 Scalability Issues

Computing exact Data Shapley values, as defined in Equation (2), is intractable because the number of subsets to process is exponential in $|\mathcal{D}|$. This renders exact computation infeasible for real-world datasets with millions of interactions. Even worse, each subset S to be processed requires the re-training and evaluation of the underlying recommendation model.

3.2.1 TMC-Shapley. Computing the Data Shapley value ϕ_i for a data point $x_i \in \mathcal{D}$ can be formulated as an expectation calculation problem: $\phi_i = \mathbb{E}_{\pi \sim \Pi}\left[V(S_\pi^i \cup \{\mathbf{x}_i\}) - V(S_\pi^i)\right]$, where Π denotes the uniform distribution over all $n!$ permutations of the data points in $\mathcal{D}$ and S_π^i is the set of all data points coming before $\mathbf{x}_i$ in the permutation π [12]. For this formulation, Ghorbani and Zou [12] present the TMC-Shapley algorithm (see Algorithm 1), which conducts a Monte Carlo (MC) estimation of the Data Shapley values.[2] TMC-Shapley repeatedly samples a permutation π from Π, computes the marginal contribution $V(S_\pi^i \cup \{\mathbf{x}_i\}) - V(S_\pi^i)$ of a data point $\mathbf{x}_i$ over S_π^i in Lines 10 and 11, and iterates through all n data points in the permutation (Line 6). Furthermore, it applies a truncation technique, by stopping to compute marginal contributions once the obtained utility is within a threshold of the utility value $V(D)$ of the whole dataset (Line 7). A convergence test is performed by checking if the mean absolute percentage deviation of the Shapley value for all data points is below a given threshold.

3.2.2 Scalability issues in TMC-Shapley. Even Monte Carlo-based algorithms such as TMC-Shapley are difficult to scale to larger datasets [27], as the number of models to retrain and evaluate is linear in the number of data points in $\mathcal{D}$, which is still not feasible for large datasets, where a single training run can take multiple hours.

[2]Note that we base Algorithm 1 on the authors' actual code from https://github.com/amiratag/DataShapley, which slightly differs from the formulation in their paper.

Algorithm 1 TMC-Shapley algorithm as proposed in [12].

```
function TMC-SHAPLEY(𝒟, V)
  φ = {φ_1, ..., φ_n} ← 0
  while not converged :
    π ← random permutation of data points in 𝒟
    v_prev ← V(∅)
    for j ∈ 1 ... n :                                   // Iterate through permutation
      if |V(D) − v_prev| > performance tolerance        // Truncation
        S^i_π ← set of data points before position j in π
        x_i ← data point at position j in π
        v ← V(S^i_π ∪ {x_i})                            // Retrain and evaluate model
        φ_i ← φ_i + (v − v_prev)                        // // Update marginal contribution
        v_prev ← v
  t ← number of permutations evaluated
  φ ← (1/t) φ                                           // Normalise by number of iterations
  return Shapley values φ = {φ_1, ..., φ_n}
```

3.3 The Potential of KNN Models

Recent research details how to efficiently compute DSVs for KNN classifiers [19]. This direction is promising for recommendation scenarios as well, where KNN models have a long tradition, ranging from classical collaborative filtering [31, 41, 42] to recent state-of-the-art algorithms in session-based [6, 18, 33, 35], session-aware [28], next-basket [9, 17, 29], and within-basket recommendation [2], to applications for conversational recommendation [56]. However, existing approaches [19, 21] for efficiently computing DSVs for KNN classifiers are not directly applicable to KNN-based recommender systems. This is because they make several assumptions about the utility function. First, they assume that the model quality metric treats the contributions of the nearest neighbors independently, which is not the case for ranking-based metrics. Second, the computational efficiency bounds in [19, 21] are derived for classification tasks with a small number of class labels, which is not the case in recommender systems, which have to rank millions of items.

3.3.1 Research question. This leads us to the main research question of this paper: *Can we efficiently compute Data Shapley values for KNN-based recommenders on large sparse interaction datasets?*

4 KMC-Shapley

Our goal is to design a general DSV estimation algorithm applicable to a wide range of recommendation models from the KNN family. In order to achieve this, we first define an abstract computational model capturing the general structure of KNN algorithms on sparse data in Section 4.1. Based on this, we design the KMC-Shapley algorithm, a specialized variant of TMC-Shapley (Algorithm 1), that skips expensive utility computations when the marginal contribution is known to be zero. We discuss the characteristics of KNN models that enable this in Section 4.3, and present the actual algorithm in Section 4.4.

4.1 An Abstract Computational Model for Neighborhood-Based Recommendation

In the following, we discuss an abstract computational model that allows us to compute data importance in a general manner for KNN algorithms.

4.1.1 Sparse representation of interactions. Each set of interactions in the training data is encoded as a sparse representation $\mathbf{x}_i$. Thereby, the training data of observed interactions is turned into a

retrieval corpus $\mathcal{D} = \{\mathbf{x}_i | \ i \in E\}$, where E denotes a use-case specific entity of interest (e.g., the ratings of a user, the ratings of an item, browsing sessions, shopping baskets, ...).

4.1.2 Retrieval-based inference. KNN models can be viewed as a combination of a retrieval function f_{ret} to query the retrieval corpus $\mathcal{D}$ and a prediction function f_{pred} to generate recommendations based on the retrieved representations. At inference time, KNN models compute recommendations for a query $\mathbf{x}_q$ in two stages:

(1) *Retrieval.* The k nearest neighbors $\mathbf{x}_{\alpha_1}, \ldots, \mathbf{x}_{\alpha_k}$ for the queried representation $\mathbf{x}_q$ are retrieved via $f_{ret}(\mathbf{x}_q, \mathcal{D}, k)$, according to a model-specific ranking function.
(2) *Item scoring.* The top items to recommend are computed via the prediction function $f_{pred}(\mathbf{x}_q, \{\mathbf{x}_{\alpha_1}, \ldots, \mathbf{x}_{\alpha_K}\})$, based on the query representation $\mathbf{x}_q$ and the retrieved neighbor representations $\mathbf{x}_{\alpha_1}, \ldots, \mathbf{x}_{\alpha_k}$.

4.2 Mapping to Concrete Recommendation Algorithms

Note that our outlined computational model is instantiated differently for different KNN algorithms. We detail a selection of examples in the following.

4.2.1 Session-based recommendation (SBR). For SBR, we discuss the recently proposed VMIS-KNN algorithm [24, 34], which is the basis for Bol's product page recommendations. In this algorithm, the interaction sequences are represented as sparse binary vectors in item space, and the retrieval function f_{ret} conducts a nearest neighbor search based on a custom, non-symmetric similarity function that computes the item overlap between a query session and the sessions in the training data, and weighs matches based on the positions of overlapping items. The item scoring function f_{pred} aggregates the top neighbors, weighted by several custom factors such as the the position of the first item match between a neighbor and the query session and the "inverse document frequencies" of items in the session.

4.2.2 Next-basket recommendation (NBR). For NBR, TIFU-KNN [17] is a popular algorithm, a variant of which is also in production usage at online grocery shopping platforms in Europe [51]. TIFU-KNN conducts a two-level hierarchical aggregation of groups of shopping baskets into a sparse vector $\mathbf{x}_u$ in item space, which represents the purchase history of a user u. The baskets $[\mathbf{s}_0^u, \ldots, \mathbf{s}_n^u]$ of a user u are first partitioned into groups of m elements, each of which is aggregated into a group vector $\mathbf{v}_b^u = \frac{1}{m} \sum_{i=0}^{m} r_b^{m-i} \mathbf{s}_{b+i}^u$, where b denotes the start index of the group, and r_b is a temporal decay rate to down-weight the influence of older baskets. The final representation $\mathbf{x}_u = \frac{1}{g} \sum_{i=0}^{p} r_g^{g-i} \mathbf{v}_{b+i}^u$ of a user u is computed via a subsequent aggregation of the g group vectors with another temporal decay rate r_g. The retrieval and item scoring functions are conceptually simple, e.g., f_{ret} may conduct a nearest neighbor search based on the Jaccard similarity and f_{pred} computes a linear combination $w\,\mathbf{x}_u + (1-w) \sum_{i=1}^{k} \mathbf{x}_{\alpha_i}$ of the user's own vector representation $\mathbf{x}_u$ with the neighbor representations $\mathbf{x}_{\alpha_1}, \ldots, \mathbf{x}_{\alpha_k}$, where w denotes the weight for the personal purchases.

4.2.3 Within-basket recommendation. This task is a variation of NBR, where a subset of the items in the basket to predict are already known. It has been introduced together with the PerNIR model [2]. PerNIR computes a sparse representation $(\mathbf{u}_u, \mathbf{C}_u)$ for the interaction history of each user u, where the first component $\mathbf{u}_u \in \mathbb{R}^{|I|}$ represents the personal consumption pattern of a user u and the second component $\mathbf{C}_u \in \mathbb{R}^{|I| \times |I|}$ represents the item co-occurrence pattern in their shopping baskets. These components are computed via a temporally weighted aggregation of the user's past shopping baskets. Let $\mathbf{h}_u$ denote the sequence of historical shopping baskets for user u, and let $\mathbf{s}_t$

denote the t-th basket in this sequence:

$$\mathbf{u}_u = \sum_{t=0}^{|\mathbf{h}_u|-1} \frac{1}{|\mathbf{h}_u| - t}\, \mathbf{s}_t \text{ and } \mathbf{C}_u = \left[\sum_{t=0}^{|\mathbf{h}_u|-1} \frac{1}{|\mathbf{h}_u| - t} \; \frac{1_{\{i_i \in \mathbf{s}_t\}}\, 1_{\{i_j \in \mathbf{s}_t\}}}{|I(i_i, \mathbf{s}_t) - I(i_j, \mathbf{s}_t)|} \right]_{ij}. \tag{3}$$

The indexes i and j refer to items here and the function $I(i_i, \mathbf{s}_t)$ returns the position of item i_i in basket $\mathbf{s}_t$. The retrieval function f_{ret} searches for the top-k similar users based on the cosine similarity $\mathbf{u}_q^\top \mathbf{u}_u \,/\, (|\mathbf{u}_q||\mathbf{u}_u|)$ between the personal consumption patterns $\mathbf{u}_q$ and $\mathbf{u}_u$ of the query user q and each other user u. During item scoring, the f_{pred} function of PerNIR computes a linear combination of the query user's personal vector $\mathbf{u}_q$ with the personal vectors of its neighbors as well as a linear combination of the query user's item co-occurrence matrix $\mathbf{C}_q$ with the item co-occurrence matrices of its neighbors. The resulting co-occurrence matrix is multiplied with a "selection and summation" vector built from the query user's evolving shopping basket $\mathbf{s}_{n+1}^q$ to account for the set of items already present. Note that N_q refers to the set of neighbors for q retrieved by f_{ret} and that w and v are hyperparameters to weight the individual components of PerNIR:

$$w \left[v\, \mathbf{u}_q + \frac{1-v}{|N_q|} \sum_{u \in N_q} \frac{\mathbf{u}_q^\top \mathbf{u}_u}{|\mathbf{u}_q||\mathbf{u}_u|}\, \mathbf{u}_u \right] + (1-w) \left[v\, \mathbf{C}_q + \frac{1-v}{|N_q|} \sum_{u \in N_q} \frac{\mathbf{u}_q^\top \mathbf{u}_u}{|\mathbf{u}_q||\mathbf{u}_u|}\, \mathbf{C}_u \right] \left[\frac{1_{\{i_i \in \mathbf{s}_{n+1}^q\}}}{|\mathbf{s}_{n+1}^q| - I(i_i, \mathbf{s}_{n+1}^q)} \right]. \tag{4}$$

4.3 Data and Model Characteristics in Neighborhood-Based Recommendation

We discuss the data and model characteristics in neighborhood-based recommendation on sparse data, which allow us to skip the expensive utility computation in cases where we already know that the marginal contribution is zero.

4.3.1 Sparse retrieval. KNN algorithms typically operate on sparse interaction data between users and items. A common characteristic of these datasets is that they contain a wide selection of items, while each user only interacts with a tiny fraction of the available items. For example, the median session length in click datasets in e-commerce ranges from two to four clicks, even in scenarios with more than a million distinct items [24], and shopping baskets in purchase datasets contain less than ten items on average, even though there are tens of thousands of distinct items available [17]. As a consequence, the resulting interaction data is extremely sparse. To account for this, KNN algorithms use sparse representations for the interaction histories and apply a sparse retriever f_{ret} with a similarity function $sim(\mathbf{x}_q, \mathbf{x}_i)$ to rank a representation $\mathbf{x}_i$ in the retrieval corpus with respect to a query $\mathbf{x}_q$. These retrievers typically ignore pairs $(\mathbf{x}_q, \mathbf{x}_i)$ of representations with no overlap in item interactions, meaning that $\mathbf{x}_i$ will never be in the neighbor set of $\mathbf{x}_q$ in that case.

4.3.2 Locality. The prediction function f_{pred} only uses the k representations with the largest similarities returned by f_{ret}. Let $\Gamma_q(S)$ denote the k-th largest similarity score between the validation sample $\mathbf{x}_q$ and the representations from a set S. If $sim(\mathbf{x}_q, \mathbf{x}_i) < \Gamma_q(S)$, then adding $\mathbf{x}_i$ to S has no impact on the utility for $\mathbf{x}_q$. The sparsity of the interaction data and the locality property of KNN models allow us to work with the much smaller set of neighbors of each validation sample instead of having to process all data points from the training data in each iteration. Apart from being able to skip certain utility computations, we can further accelerate the algorithm.

4.3.3 Additivity of utilities. Analogous to classification and regression scenarios, the utility V of a model with respect to a validation dataset $\mathcal{D}_{val}$ in recommendation scenarios is computed by averaging the utilities for the individual validation samples $\mathbf{x}_q \in \mathcal{D}_{val}$ for common metrics like NDCG, MRR, Recall, etc. This additional property of V leads to $V(S_\pi^i \cup \{\mathbf{x}_i\}) - V(S_\pi^i) =$

Algorithm 2 KMC-Shapley algorithm.

```
1  function KMC-SHAPLEY(𝒟, V, k, 𝒟_val)
2    ϕ = {ϕ_1, ..., ϕ_n} ← 0                                  // Initialize Data Shapley values
3    N ← ∅                                                    // Initialize neighbor index
4    for x_q ∈ 𝒟_val :                                        // Populate index with neighbors
5      N_q = {(x_α1, s_α1), ..., (x_αM, s_αM)} ← f_ret(x_q, 𝒟)
6    while not converged :
7      π ← random permutation of data points in 𝒟
8      parfor x_q ∈ 𝒟_val                                     // Iterate over validation samples in parallel
9        Sort N_q according to the positions in π
10       h ← min-heap of capacity k                           // Initialize min-heap
11       Initialize pre-aggregate σ_q
12       v_prev ← 0                                           // Initialize previous utility
13       for (x_αi, s_αi) ∈ N_q :
14         if |h| < k
15           insert (x_αi, s_αi) into h
16           Include x_αi into pre-aggregate σ_q
17         else
18           (x_h, s_h) ← data point and similarity of heap root
19           if s_αi > s_h
20             Remove x_h from pre-aggregate σ_q
21             Include x_αi into pre-aggregate σ_q
22             update heap root of h with (x_αi, s_αi)
23       if h changed                                         // Set of k-nearest neighbors changed
24         v ← V_q(f_pred(x_q, σ_q))                          // Utility with current neighbors
25         j ← position of x_αi in π
26         ϕ_πj ← ϕ_πj + (v − v_prev)                         // Update Shapley value
27         v_prev ← v                                         // Update previous utility
28
29   t ← number of permutations evaluated
30   ϕ ← (1/t) ϕ                                              // Normalise by number of iterations
31   return Data Shapley values ϕ = {ϕ_1, ..., ϕ_n}
```

$\frac{1}{|\mathcal{D}_{val}|} \sum_{\mathbf{x}_q \in \mathcal{D}_{val}} V_q(S^i_\pi \cup \{\mathbf{x}_i\}) - V_q(S^i_\pi)$, where V_q computes the utility with respect to the validation sample $\mathbf{x}_q$. This enables a mapreduce-like parallelisation pattern, where we process each individual validation sample $\mathbf{x}_q \in \mathcal{D}_{\text{val}}$ in parallel and aggregate the resulting marginal contributions per training data point subsequently.

4.3.4 Linear aggregations at inference time. The prediction function f_{pred} in many KNN models first conducts a linear aggregation of the retrieved representations (e.g., a weighted sum of their sparse vector representations) and subsequently selects the items to recommend via a non-linear operation. Since the MC procedure requires us to sequentially scan the permutation of training points and investigate the impact of an additional sample (Lines 10–11 in Algorithm 1), it will repeatedly invoke f_{pred} with one new neighbor and $k-1$ neighbors that have been seen in the previous step. This makes it possible to reuse the aggregation result of the $k-1$ neighbors from the previous step. In order to achieve this, we make the KNN model maintain an aggregate σ_q of the current top-k neigbor set and directly compute predictions from this via $f_{pred}(\mathbf{x}_q, \sigma_q)$, which allows us to avoid repeating redundant aggregations.

4.4 KMC-Shapley Algorithm

Based on the outlined characteristics, we present *KMC-Shapley* in Algorithm 2. This variant of TMC-Shapley is tailored for KNN models and runs several orders of magnitude faster (as we experimentally show in Section 5.1.1).

KMC-Shapley starts with retrieving and indexing the top-M neighbors $\{\mathbf{x}_{\alpha_1}, \ldots, \mathbf{x}_{\alpha_M}\}$ of each validation sample $\mathbf{x}_q$ in Lines 4–5, where s_{α_i} denotes the similarity $sim(\mathbf{x}_q, \mathbf{x}_{\alpha_i})$. The parameter M denotes the maximum number of neighbors to consider per validation sample and is typically set to a high number. This parameter is inspired by recent statistics research which shows that high-cardinality subsets (with more than 100 elements) produce unreliable estimates of the marginal contribution in DSV computations [27]. Analogous to TMC-Shapley, our algorithm runs several MC iterations and generates a random permutation π of the data points in $\mathcal{D}$ in each iteration (Line 7). In contrast to TMC-Shapley, KMC-Shapley does not iterate over the data points in $\mathcal{D}$, but over each validation sample $\mathbf{x}_q$ in parallel (Line 8).

For each validation sample $\mathbf{x}_q$, our algorithm needs to consider its indexed neighbors only, according to their order in the permutation π (Line 9), as only the addition of new neighbors can produce a non-zero marginal contribution for $\mathbf{x}_q$. Our algorithm iterates through these neighbors (Line 13) and maintains the set of top-k neighbors in a binary heap (Lines 15 and 22). It simultaneously maintains the corresponding pre-aggregate σ_q of the top-k neigbor set under changes (Lines 16, 20 and 21). The computation of the marginal contribution of adding a new neighbor $\mathbf{x}_\alpha$ is only necessary (e.g., potentially non-zero) if the top-k neighbor set for $\mathbf{x}_q$ changes. In such cases, the change in utility is computed and used to update the DSV estimate corresponding to $\mathbf{x}_\alpha$ (Lines 23–27). Finally, the Data Shapley values are normalized by the number of iterations conducted and returned (Lines 29–31).

4.4.1 Complexity analysis. Let P denote the number of permutations considered. Algorithm 2 runs P iterations over $|\mathcal{D}_{\text{val}}|$ validation samples and has to process at most M neighbors per validation sample. Sorting these according to their positions in the permutation π can be done in $O(M \log M)$. For each neighbor, there is a cost of $\log k$ for a potential update of the heap and a constant cost of a single summation and subtraction for maintaining the pre-aggregate σ_q. Since $M \gg k$, this results in an overall time complexity of $O(P\,|\mathcal{D}_{\text{val}}|\,M \log M)$. These optimizations make KMC-Shapley significantly more scalable than the baseline, even for large validation sets and neighborhood sizes.

4.4.2 Toy example. We visualize the advantages of KMC-Shapley over TMC-Shapley on a toy example in Figure 2. For that, we show how both algorithms process a permutation π of a dataset set $\mathcal{D}$ with eight elements $\mathbf{x}_1, \ldots, \mathbf{x}_8$ for a KNN model with $k = 2$ and a single validation sample $\mathbf{x}_q$. The operations shown correspond to Lines 4–10 in Algorithm 1 for TMC-Shapley and to Lines 9–24 in Algorithm 2 for KMC-Shapley. This setup highlights how the sparsity and locality properties of KNN models reduce computation overhead, even in simple cases.

On the left side, TMC-Shapley enumerates eight subsets of $\mathcal{D}$ according to the permutation π and evaluates the utility V of the resulting predictions for the validation sample $\mathbf{x}_q$ each time. KMC-Shapley (on the right side) takes the similarities of the data points in D to the validation sample $\mathbf{x}_q$ into account and directly maintains the top-2 neighbor set of $\mathbf{x}_q$ (highlighted in blue). This allows our algorithm to skip redundant computations, since only changes in the top-2 neighbor set of $\mathbf{x}_q$ will lead to changes in utility. Concretely, KMC-Shapley skips the utility computations for adding the data points $\mathbf{x}_5$, $\mathbf{x}_4$, $\mathbf{x}_1$ and $\mathbf{x}_6$, which have a similarity of zero to the validation sample $\mathbf{x}_q$. Moreover, KMC-Shapley can skip the utility computation for adding $\mathbf{x}_8$, since the similarity of $\mathbf{x}_8$ to $\mathbf{x}_q$ is too small to change the top-2 neighbor set. Overall, we see that KMC-Shapley can significantly reduce the number of required utility evaluations.

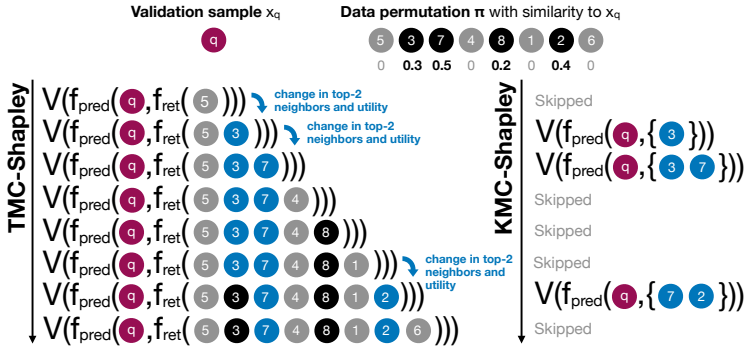


Fig. 2. Comparison of TMC-Shapley to KMC-Shapley for a toy example with eight data points, a single validation sample $\mathbf{x}_q$ and a model with $k = 2$. KMC-Shapley is able to skip many redundant computations, since only changes in the top-2 neighbor set of $\mathbf{x}_q$ lead to changes in utility. TMC-Shapley (left side) processes eight data subsets for the permutation π and evaluates the utility function V each time. KMC-Shapley (on the right side) takes the similarities of the data points to $\mathbf{x}_q$ into account and directly maintains the top-2 neighbor set of $\mathbf{x}_q$ (highlighted in blue). This allows KMC-Shapley to skip many redundant computations, since only changes in the top-2 neighbor set of $\mathbf{x}_q$ lead to changes in utility.

4.5 Implementation

We implement our approach with highly-optimized Rust code in the Illoominate library, see Section A in the Appendix for details.

5 Evaluation

We evaluate the efficiency and scalability of KMC-Shapley and experimentally validate that it identifies impactful data points. We evaluate on both public and proprietary datasets. For comparability and reproducibility, we choose public datasets from the publications on the neighborhood-based algorithms for which we compute Data Shapley values [17, 35]. The proprietary Bol datasets complement these benchmarks by allowing us to assess scalability and practical data-debugging behavior in large real-world e-commerce settings with millions of click and purchase interactions. We focus on two recommendation tasks in e-commerce, where KNN models provide competitive performance [29, 34]: session-based recommendation [24, 34], where the goal is to predict the next item that a user will click on, and next-basket recommendation [2, 17, 29], where the goal is to predict the items in the next shopping basket purchased by a user. Note that we run KNN-based recommenders in production for these tasks [22, 24, 51].

5.1 Efficiency & Scalability

5.1.1 Efficiency. The goal of our first experiment is to showcase that our proposed optimizations (Sections 4.3 and 4.4) for KMC-Shapley drastically reduce the runtime compared to TMC-Shapley.

Experimental setup. We run this experiment on a Windows 10 machine with an Intel i9-10900KF CPU. We use a sample of a public session dataset with 250,000 clicks as training data and 14,058 clicks as validation data, compute DSVs for session-based recommendation with a variant of the VS-KNN recommendation algorithm [24, 34], and repeat each run five times for $k \in \{50, 100, 250, 500\}$. We measure the mean runtime for a single MC iteration over all training data points with different algorithm variants. We design this experiment as an ablation study for the proposed optimizations originating from Section 4.4, based on the characteristics outlined in Section 4.3. For that, we introduce our proposed optimisations step by step to transform TMC-Shapley into KMC-Shapley.

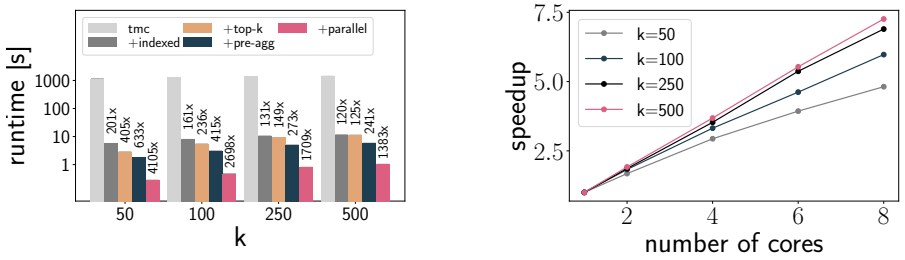


Fig. 3. Efficiency and scalability of KMC-Shapley: each optimization reduces the runtime (left) and the algorithm's runtime scales linearly with the number of cores (right).

The baseline `tmc` denotes a Rust implementation of the vanilla TMC-Shapley algorithm (Algorithm 1), which retrains the recommendation model each time. Next, `+indexed` denotes a variant that does not retrain the KNN-SR model, but reuses the indexed neighbors per validation sample instead; the `+top-k` variant additionally maintains the top-k neighbor set in a binary heap and only computes marginal contributions once this set changes; the `+pre-agg` variant additionally maintains a pre-aggregate for accelerating inference and the final optimisation `+parallelism` parallelises the computation with eight threads, and corresponds to the fully optimised KMC-Shapley implementation.

Results and discussion. We plot the resulting mean runtimes (and the speedup over the `tmc` baseline) on a logarithmic scale on the left of Figure 3. We observe that all our performance optimisations are beneficial, as each optimisation further reduces the runtime. In this experiment, KMC-Shapley is three orders of magnitude faster than the vanilla TMC-Shapley implementation which repeatedly retrains the model. As expected, the largest runtime reduction originates from reusing the neighbor sets and avoiding model retraining in the `+indexed` variant (which exploits the "sparse retrieval" and "locality" characteristics outlined in Section 4.3). In this experiment, we find that KMC-Shapley can conduct a single iteration for all training data points in a second or less for all values of k. The results confirm that it is indeed possible to design customized, highly accelerated variants of the Data Shapley value computation for nearest neighbor models on sparse data.

5.1.2 Scalability. The goal of the next experiment is to show that our implementation benefits from additional computational resources, such as CPU cores.

Experimental setup. We experiment with a session-based recommender using a variant of the VS-KNN algorithm [24, 34] on a large sample of 10,431,353 clicks in 1,668,295 sessions from Bol. We run KMC-Shapley on this data with a validation set containing 250,000 clicks in 40,023 sessions, vary the number of neighbors k, and increase the number of cores from one to eight. We execute this experiment on a machine with a Mac M1 Pro, repeat each run six times, and report the speedup over the single-threaded baseline.

Results and discussion. We plot the resulting speedups on the right of Figure 3. We observe that the runtime scales linearly with the number of available cores, which is expected due to the map-reduce-like computational pattern in KMC-Shapley (Section 4.3). We observe diminishing returns for smaller values of k, which we attribute to the fact that the algorithm is less dominated by the computational efforts for inference in these cases.

Even on this large dataset of over 10 million clicks, a single KMC iteration with eight cores only takes 136 seconds on average for all training data points.

5.2 Impact of Data Removal

Next, we evaluate how well KMC-Shapley identifies impactful data points for interaction data. For that, we adopt a common data removal experiment [12, 21, 27] to our recommendation setup. In this experiment, data points are removed from the training data according to a given order (e.g., sorted ascendingly or descendingly with respect to their importances), and the prediction quality of a model trained on the remaining data is repeatedly evaluated on a held-out test set. The rate at which the prediction quality changes indicates how impactful the chosen removal order is.

5.2.1 Session-based recommendation. We conduct this data removal experiment for session-based recommendation [6, 34].

Experimental setup. Concretely, we use a variant of the VS-KNN recommendation algorithm [24, 34] with hyperparameters $k = 50$ and $M = 500$. We experiment with samples from two public datasets (nowplaying, instacart) and three proprietary click datasets (bol-literature, bol-babycare, bol-woodentoys) with samples of up to 1.7 million clicks from different product categories on Bol. We prepare the datasets as follows: we conduct a temporal split of the sessions into training sessions and held-out sessions, and we randomly split the held-out sessions into 50% validation and 50% test data. We make sure that we only retain sessions with items seen in the training data (e.g., we ignore cold-start items for which no clicks have been seen yet). Table 1 summarises the statistics of the resulting datasets. Note that it is crucial to choose a large enough validation set to avoid a high variance in the DSV scores; this can be determined upfront via preliminary experiments with differently sized validation sets and different random seeds.

Table 1. Datasets for session-based recommendation.

				#clicks		
Dataset		**#sessions**	**#items**	training	validation	test
bol-literature	proprietary	393,655	50,202	1,704,661	168,109	166,289
bol-babycare	proprietary	29,324	3,161	127,088	13,351	13,395
bol-woodentoys	proprietary	98,420	8,937	437,329	44,427	44,825
nowplaying	public	97,922	196,531	489,367	58,277	57,616
instacart	public	21,068	18,661	124,376	60,104	59,363

We compute Data Shapley values via KMC-Shapley and LOO errors (as defined in Section 3.1 with the computation detailed in Section B) with respect to the MRR for the first 20 recommended items (referred to as MRR@20). We evaluate the impact of removing highly important sessions with positive DSVs in descending order. We repeat this analogously for sessions with positive leave-one-out errors and also include two baselines, one that simply removes data points at random, and another heuristic baseline that considers the number of clicks contained in a session (based on the assumption that very long sessions are likely to originate from bots and crawlers). We replicate this experiment for removing low-scored data, where we remove the sessions with negative DSVs and LOOs in ascending order instead.

Results and discussion. Figure 4 shows the absolute differences in MRR@20 on the held-out test data when removing up to 25% of the training sessions. Each experiment is repeated three times, and we report mean values as solid lines and standard deviations as shaded areas. When removing highly important data, we observe clear differences across the methods. Removing sessions ranked by Data Shapley values (DSV) causes the steepest decline in MRR@20, up to 0.06, demonstrating

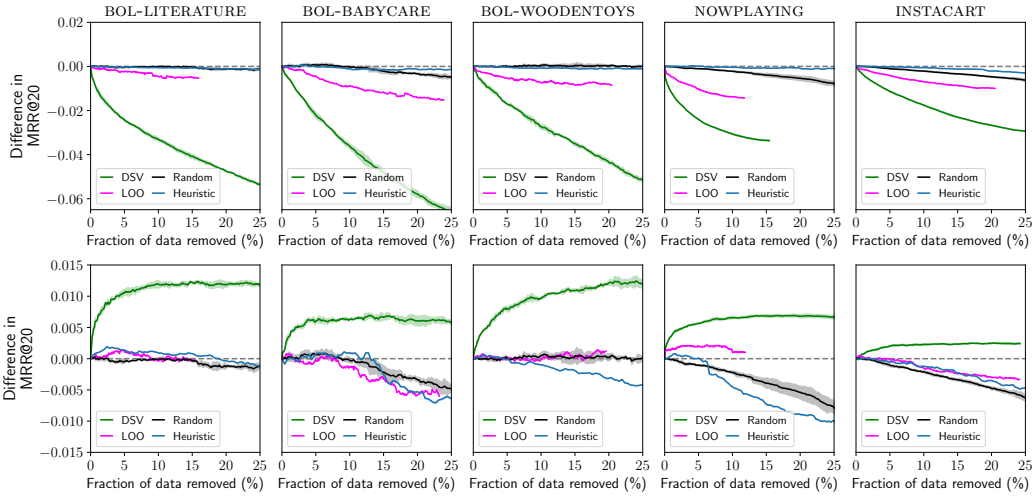


Fig. 4. Impact of removing the most important data points (top row) and data points with negative importance scores (bottom row) from a session-based recommender system across datasets. Data identified by the Data Shapley value (DSV) has stronger impact than data identified by leave-one-out error (LOO), random removal, or session length as a simple heuristic.

that KMC-Shapley more effectively identifies high-impact training data than LOO, random removal or removal based on the heuristic. When removing sessions with negative importance scores, DSV again yields the largest improvements across datasets, confirming that these sessions often represent harmful or noisy data. LOO-based removal improves performance less consistently, and removal based on random choice shows only minor effects in either direction. Removing up to 5% of the longest sessions slightly outperforms random removal, but performance degrades beyond that. This highlights the limited effectiveness of rule-based heuristics compared to learned data importance scores. We now evaluate scalability on real-world datasets. For this experiment, we use a Mac M3 Pro with 6 cores, on which the DSV computation requires between 1,420 to 2,481 iterations to converge and the time per iteration varies from 0.7 seconds on BOL-BABYCARE to 21.4s on INSTACART. Even for the largeset dataset BOL-LITERATURE with more than two million clicks, the computation finishes in under 10 hours.

5.2.2 Next-basket recommendation. We repeat the data removal experiment for next-basket recommendation.

Experimental setup. The setup is analogous to the previous experiment with the following differences. We use the TIFU-KNN model [17], which is deployed in production on online grocery shopping platforms of brands from our partner companies [51]. We use NDCG as evaluation metric [3], which is common for this recommendation task. For the proprietary BOL-PURCHASES dataset, we report NDCG@21 because this matches the user interface in our production platform, where recommendations are displayed in three columns and seven rows, resulting in 21 visible recommendation slots. For the public TAFENG dataset, we report NDCG@20 to remain comparable with previous work. We experiment on a large proprietary dataset called BOL-PURCHASES containing a sample of more than two million purchase events from Bol. Additionally, we experiment with a sample of 1,000 users from the public dataset TAFENG. Table 2 presents the dataset characteristics. We use the purchase history of each user as training data, from which we hold-out their most recent shopping basket. We randomly split these most recent baskets into validation and test data.

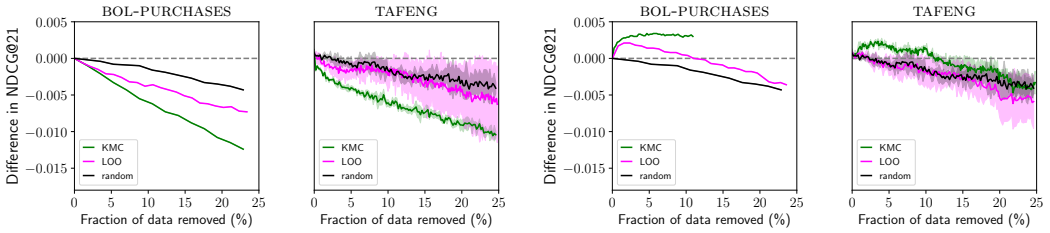


Fig. 5. Impact of removing purchase histories in NBR, with removal of high-value histories on the left and removal of histories with negatives scores shown on the right. We report differences in NDCG@21 for BOL-PURCHASES and NDCG@20 for TAFENG. Data identified by KMC-Shapley has stronger impact than data identified by the leave-one-out error (LOO) or randomly removed data.

Note that we remove user histories from the training data (analogous to the previous experiment) so that they will not occur in the neighbor sets, but still use a user's personal history for their personalized prediction [29].

Table 2. Datasets for next-basket recommendation.

Dataset		#users	#purchases	#baskets	#items
BOL-PURCHASES	proprietary	97,867	2,309,445	417,028	376,672
TAFENG	public	1,000	41,421	6,586	6,586

Results and discussion. In Figure 5 we plot the resulting differences in NDCG after removing up to 25% of the training sessions. The results observed are in line with the results from the previous experiment on session-based recommendation. Removing the training sessions according to their Data Shapley values drastically reduces the scores, which indicates that KMC-Shapley performs significantly better at identifying high impact data points. Removing training sessions according to the LOO error still reduces the scores faster than random removal, but the corresponding sessions have a lower impact on scores, compared to removal by Data Shapley values.

6 Applications

We discuss some applications of data importance at Bol.

6.1 Identifying Outliers and Corrupted Data

The main purpose of data importance is to help us find corrupted and harmful interaction data in the production recommender system [24] of Bol. This session-based recommender serves the "others also viewed" recommendations on the product pages. Apart from improving our recommender systems, uncovering data issues is also important for other teams, e.g., those responsible for product quality and risk issues on the platform.

In order to showcase this use case, we draw a large sample of historical click data from the babycare, wooden toys, and wooden pencils categories, and compute DSVs with MRR@20 as the target metric of a variant of our VS-KNN recommendation algorithm [24, 34]. We find that the lowest-scored sessions contain inconsistencies, problematic products, and corrupted data. We show three example sessions taken from the ten lowest-scored sessions found in these data samples in Figure 6. All of these sessions are harmful to the recommender system (i.e., including them in the training data negatively impacts prediction quality) and suffer from data issues. The top-most session originates from a user exploring baby bath products and for example contains a foldable laundry

basket, which has been incorrectly advertised as a baby bath by its seller, and which according to its reviews is actually dangerous to use for bathing children. The remaining two sessions shown in the figure suffer from different issues. The second session contains pencil products, many of which suffer from quality issues in their metadata, such as pixelated and unidentifiable images. The third session contains a massive number of 83 clicks on wooden toys, and the toys target highly variable age ranges (one-year-olds to eight-year-olds), making this session unsuitable as a basis for recommendation. These examples showcase that DSVs identify low-quality interaction data with issues that are hard to anticipate upfront.

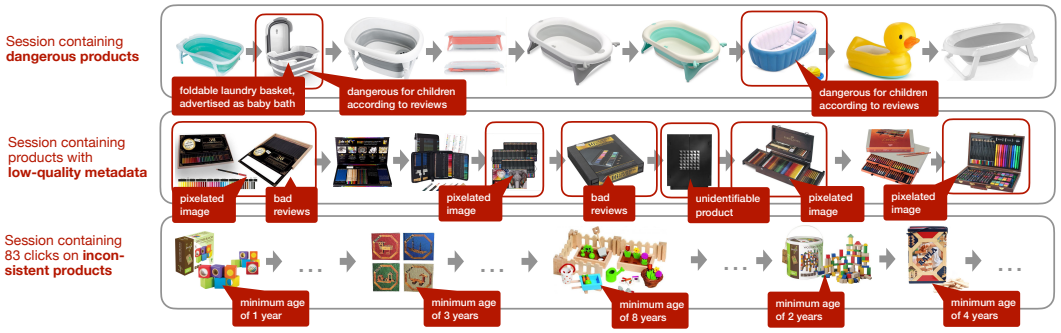


Fig. 6. Examples of sessions with negative importance scores in Bol's click data (*each box represents a single session consisting of a sequence of clicks on items, arrows show the order of clicks*). We encounter sessions containing dangerous products (e.g., a foldable laundry basket incorrectly advertised as baby bath), sessions with metadata quality issues (e.g., pixelated and unidentifiable images) as well as sessions with inconsistent products (wooden toys for different age ranges).

We repeat this analysis for a large sample of purchase histories from Bol and a next-basket recommendation model [17] (a variant of which we run in production as well), using NDCG@20 as the target metric. We again find that the lowest-scored purchase histories contain data that is not helpful for our recommender systems, and refer to Figure 7 for examples. The histories contain electronics items bought in unreasonable numbers (e.g., over a hundred PlayStation controllers), switch to completely unrelated product categories (coffee, dental products) at some point, and are therefore clearly uninformative for a recommender system. We attribute such purchase histories to commercial accounts, which buy large numbers of products for several thousands of Euros as a response to price promotions, probably intended for reselling.

Fig. 7. Examples of purchase histories with negative importance scores from Bol (*boxes indicate items bought together in a shopping cart, arrows point to the next shopping cart*). The histories contain electronics items bought in unreasonably high numbers (e.g., over a hundred PlayStation controllers), an activity gap of more than half a year, and switch to unrelated product categories (coffee, dental products) at some point.

We also investigate the sessions and purchase histories with the highest DSVs from this experiment and plot examples in Figure 8 and Figure 9. We find that the DSVs identify high-value

sessions with consistent product selections in click data, and high-value shopping baskets with high turnover items (baby formula, toilet paper) in purchase history data.

Fig. 8. High-value sessions with consistent item selections.

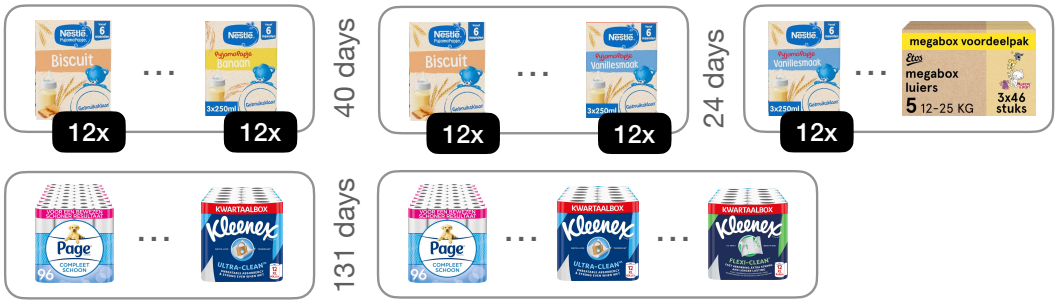


Fig. 9. High-value purchase histories with high turn-over items.

6.2 High-level Understanding of Interaction Data

Apart from inspecting individual browsing sessions or purchase histories, DSVs also contribute to a high-level understanding of the interaction data in our recommender systems. To give an example, we group the sessions from the datasets in Section 5.3 by their age in days and their length (the number of items), and plot these against the normalised mean Data Shapley values of the sessions in Figure 10. We can observe that sessions from the days before the prediction time are clearly most important for the recommender systems. Sessions that are between three weeks and eight weeks old have lower but constant importance, and the impact starts to decline afterwards. This is in line with general observations about the focus on timely data made in recent papers [34]. Moreover, we observe that the importance of sessions grows with their length, but only up to a certain point (which differs per category). This insight helps with designing features for other use cases on interaction data, e.g., detecting bots visiting the website.

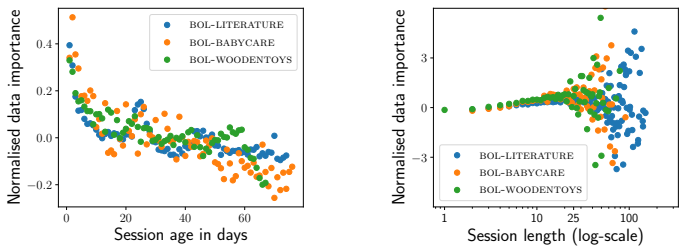


Fig. 10. Relationship of the data importance of sessions to their age (in days) and length (in number of items) for various product categories.

6.3 Increasing the Sustainability of Recommendations via Data Pruning

A major goal of Bol is to make sustainable shopping easier for customers.[3] We present an experimental application of DSVs, which contributes to this mission. The aim here is to increase the number of sustainable items in the predictions of our session-based recommender, without compromising the prediction quality. We choose a data-centric approach that prunes the underlying data in an effort to remove the histories containing unsustainable products. This allows us to maintain the prediction quality of our recommender system by letting the real histories with a higher number of sustainable products serve as a basis for future recommendations.

Operational definition of sustainability. In this experiment, we use sustainability in an operational catalogue sense. Products receive the platform-defined *Good Choice* label when they have significant sustainability benefits compared to similar items, for example because they carry an approved quality mark, consist of at least 50% recycled material, or belong to a product category approved as a more sustainable alternative.[4] This label is not inferred by our recommender system and is not generated through automated semantic analysis of product text or images. In our dataset, sustainability is represented as a binary catalogue attribute derived from the platform-defined eligibility criteria and the corresponding product metadata. For example, an FSC-certified wooden toy receives the label if the corresponding quality-mark metadata and proof are present, whereas an otherwise similar non-certified toy does not. This definition differs from recent work on green or sustainability-aware recommender systems that studies the computational carbon footprint of recommender models [47, 52] or estimates item-level product carbon footprints from product descriptions and metadata [53]. Our experiment instead studies whether data pruning can increase the exposure of products that already carry a platform-defined sustainability label, while preserving recommendation quality. To this end, we augment the product metadata with a binary flag that indicates whether an item was produced in a sustainable manner. Furthermore, we modify the MRR metric to include a sustainability term that expresses the number of sustainable products in a given recommendation. Specifically, we define a utility function, which we call *SustainableMRR@t* as:

$$\text{SustainableMRR@t} := 0.8 \cdot \text{MRR@t} + 0.2 \cdot \frac{s}{t}. \tag{5}$$

This utility combines the MRR@t with a sustainability coverage term $\frac{s}{t}$, where s denotes the number of sustainable items among the t recommended items.

For this internal experiment, we compute Data Shapley values and leave-one-out errors for our recommender system with SustainableMRR@21 as a utility on click datasets from the baby care and wooden toys categories on Bol, which contain large numbers of sustainable products. Next, we prune the training data based on the computed DSVs (by removing the 5% lowest scored data points) and evaluate the recommendations on held-out test data. We observe that removing this data significantly increases the SustainableMRR@21 metric across all cases and that both the MRR@21 as well as the sustainability coverage increase as a result of the pruning. Pruning based on LOO leads to unreliable results in our experiment since we observe a small increase for the wooden toys category, but a decrease in SustainableMRR@21 for recommendations of baby care items. In summary, these results confirm that we can use DSVs with custom-designed utility functions for the data-centric optimization of existing recommendation models.

In Figure 11, we visualise how this pruning impacts an actual recommendation: we show the top-recommended items for a session with a single click on a baby oil[5] item before and after pruning.

[3] https://over.bol.com/en/sustainability/

[4] See the public platform documentation on sustainability and the Good Choice label: https://partnerplatform.bol.com/en/idp/good-choice-label.

[5] https://www.bol.com/nl/nl/p/baby-pakket/9300000004594165/

We see that the recommendations contain two more sustainable items (care products with the "nordic ecolabel"[6]) once the training data has been optimised for sustainability. We are currently in the process of preparing an A/B test for the resulting recommender.

Fig. 11. Example for the increase in sustainable products with eco-label in the recommendations for a baby care product after optimising the training set based on a utility function incorporating item sustainability.

7 Transferability to Neural Recommendation Models

Finally, we investigate whether the DSVs estimated with KMC-Shapley on a KNN session-based model can serve as proxies for data valuation in neural recommenders. Since computing Shapley values directly for neural models is computationally prohibitive, establishing the transferability of importance scores from cheap proxy models would be a crucial step towards model-agnostic data curation [19, 21].

7.1 Experimental setup

We re-use the DSVs computed for the bol-woodentoys category in Section 5.2 and repeat the data removal experiment presented there. However, instead of evaluating the impact on the original VMIS-KNN model, we experiment with four neural session-based recommenders implemented in the RecBole library [59], selected to represent distinct architectural inductive biases: the recurrent model GRU4Rec [15], the graph-based GC-SAN [57], the sparse-interest model SINE [48], and the representation learning framework CORE [16]. We evaluate the impact of removing the most important (and least important) training sessions (as identified by KMC-Shapley on the KNN model) on the performance of these neural models.

7.2 Results and discussion

Figure 12 shows that the transfer of proxy importance rankings is architecture-dependent. In the high-value removal setting (top row), removing sessions ranked highly by the KNN proxy leads to a clear performance drop for GRU4Rec and GC-SAN, suggesting that the proxy DSVs identify training sessions that are impactful for these models. For SINE and CORE, the degradation is smaller and less consistent.

A plausible interpretation, grounded in the model architectures, is that the KNN proxy assigns high value to sessions that frequently act as nearest neighbors for many evaluation sessions and exhibit strong local co-occurrence under similarity-based retrieval. This signal may align more closely with models explicitly designed to use sequential transition structure (GRU4Rec [15]) or graph-based session transitions (GC-SAN [57]). In contrast, SINE [48] is designed to extract sparse interest representations from behavior sequences, and CORE [16] introduces architectural constraints intended to improve robustness and representation consistency, which may reduce dependence on the specific high-similarity sessions emphasized by the KNN proxy. In the low-value removal setting (bottom row), removing sessions with negative proxy DSV scores consistently improves performance across all models, indicating that proxy valuation can provide a useful signal even when the transferability of high-value rankings is weaker. Interestlingly, we observe for all models that the impact of removal based on LOO values exhibit inconsistent behavior (e.g., even

[6] https://www.nordic-swan-ecolabel.org

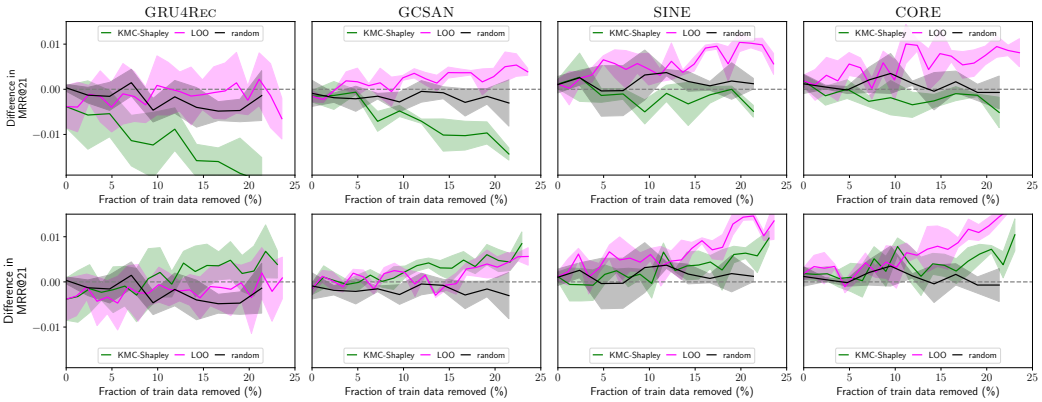


Fig. 12. Impact of data removal for neural SBR models with importance scores computed by KMC-Shapley on a KNN-SR model. For GRU4Rec and GCSAN, the proxy DSVs consistently identify impactful data points, both for removing high-value and low-value data, and outperform LOO scores. For SINE and CORE, the transferability is less clear.

removing assumingly high value sessions improves performance in some cases) with high variance, indicating that more elaborate data valuation techniques are required.

Overall, these results suggest that KMC-Shapley scores computed on a KNN proxy can provide transferable importance information for some neural recommenders, but that the quality of transfer depends on architectural choices and remains an open research question. We emphasize that this interpretation remains speculative: this initial evaluation covers only four architectures and a single proxy valuation method. Future work should systematically evaluate proxy transfer across larger sets of neural architectures, proxy recommenders, and datasets, and develop quantitative measures that predict when proxy rankings align with a neural model's learning signal. Another promising direction is to explore hybrid proxy models that combine retrieval-based similarity with learned neural representations, potentially yielding importance scores that are more robust across architectures.

8 Conclusion

We have presented a scalable algorithm for debugging interaction data of KNN-based recommender systems via the Data Shapley value, and showcased that it identifies impactful data points in datasets with millions of interactions. Compared to our prior conference version, this article substantially extends both the empirical scope and the depth of analysis.

We included a complete next-basket recommendation study, providing full data-removal results and a deeper analysis of item- and session-level effects. We further investigated the transferability of proxy-based data valuation by applying KMC-Shapley scores computed on a KNN session-based proxy to neural recommenders. Our results show strong transfer for GRU4Rec and GC-SAN, but substantially weaker effects for SINE and CORE, demonstrating that proxy-based data valuation is architecture- and regime-dependent rather than universally applicable. In addition, we expanded the analysis of session characteristics, showing how data importance relates systematically to session age and session length across multiple product categories. These findings reinforce prior academic observations on the importance of recency in training data and provide practical insight into which interactions contribute most to recommendation quality.

Finally, we demonstrated a novel application of data debugging for ecological sustainability, illustrating how pruning low-value data can increase the share of sustainable product recommendations, thereby highlighting the interpretability and business relevance of data valuation beyond offline accuracy metrics.

8.1 Limitations

Our approach is tailored to nearest-neighbor models operating on sparse interaction data and is not directly applicable to neural recommenders. Several properties exploited by KMC-Shapley, such as localized influence and explicit neighborhood structure (Section 4.3), do not hold for models based on dense representations. In addition, our results show that leave-one-out (LOO) estimates are unreliable for ranking highly influential training sessions, as they produce unstable and non-monotonic effects under high-value removal, limiting their usefulness for principled data valuation. More generally, efficiently maintaining data importance scores under continuous data growth remains an open challenge, as recomputing Shapley values from scratch is impractical for dynamic, real-world datasets.

8.2 Future work

A promising direction for future work is to investigate how data valuation could be applied to retrieval-augmented LLM recommender systems [8, 60]. Deng et al. [7] introduce a removal-based, leave-one-out-style valuation criterion that measures the utility degradation caused by removing a retrieved context. In retrieval-augmented LLM recommenders, the analogous units could be a sparse, explicitly retrieved set of recommendation-specific evidence exposed at inference time, such as retrieved candidates, collaborative-filtering signals, user-history snippets, or prompt-context items. Future work should study DSV-style valuation over such retrieved recommendation contexts, and investigate whether locality assumptions analogous to those exploited by KMC-Shapley hold here, and can reduce the required number of utility computations.

Future empirical work includes broadening the public benchmark coverage for the session-based and next-basket settings studied here, extending the evaluation to additional recommendation paradigms [2, 9, 11], incorporating alternative data importance estimation methods [27, 54], and developing incremental or streaming variants of data valuation to support continuously evolving datasets. Another promising direction is to study how temporal relevance and data weighting can be more directly incorporated into neural recommendation models, informed by the insights provided by proxy-based data valuation.

Acknowledgments

We thank our reviewers for their valuable feedback. This research was supported by Ahold Delhaize, through AIRLab Amsterdam, the Dutch Research Council (NWO), under project numbers 024.004.022, NWA.1389.20.183, and KICH3.LTP.20.006, and the European Union's Horizon Europe program under grant agreements No. 101070212 (FINDHR) and No. 101201510 (UNITE). All content represents the opinion of the authors, which is not necessarily shared or endorsed by their respective employers and/or sponsors.

A Open Source Implementation

Illoominate is implemented in Rust, with a Python frontend to facilitate its integration with data science code. The library is available at https://github.com/bkersbergen/illoominate under an open license. In order to compute data importance scores, users provide their interaction data with a validation set and specify the parameters of the recommendation scenario. For example, Listing 1

shows how to identify potentially corrupted session data, by first computing DSVs for all sessions and then retaining sessions with negative importances only.

```
import pandas as pd
import illoominate as ilm

sessions = pd.read_parquet(...)
validation = pd.read_parquet(...)

# Data Shapley values for sessions
data_shapley_values = ilm.data_shapley_values(
    train_df=sessions, validation_df=validation, model='vmis',
    metric='mrr@20', params={'m':250, 'k':250, 'seed':42})

# Problematic sessions with negative data shapley values
negative = data_shapley_values[data_shapley_values.score < 0.0]
corrupted_sessions = sessions.merge(negative, on="session_id")
...
```

Listing 1. Example code for identifying potentially corrupted session data with Illoominate.

A.1 Interaction data

Our library currently supports two types of interaction data: (*i*) *browsing sessions*, consisting of a session identifier and a list of time-stamped clicks on items; and (*ii*) *purchase histories*, consisting of a user identifier and a list of purchased shopping baskets, each represented by a set of item identifiers. This data must be provided in the form of dataframes or CSV files, which follow a simple schema: for sessions, each line represents a click with a `session_id`, `item_id` and `timestamp`; for purchase histories, each line represents a purchase event with a `user_id`, `basket_id` and `item_id`.

A.2 Recommendation models

An important design goal of our library is to abstract from the concrete instantiation of a particular KNN algorithm so that we can implement data importance algorithms once and apply them to all models from the KNN family. In our Rust code, we define the trait `RetrievalBasedModel` for all recommendation algorithms in Illoominate based on the outlined computational model. We implement the VMIS-KNN [24] and TIFU-KNN [17] algorithms accordingly.

A.3 Data importance

Illoominate currently supports computing data importance for interaction data via Data Shapley values and leave-one-out errors. All data importance algorithms are implemented in a general way via a Rust trait called `Importance` which is compatible with any recommendation model supporting the previously defined `RetrievalBasedModel` trait. We also design a general trait for utility functions and implement common ranking-based metrics in recommender systems such as Mean Reciprocal Rank (MRR), Normalised Discounted Cumulative Gain (NDCG), Recall, Precision or HitRate [3]. Our design also allows users to implement custom utility functions for their use cases.

B Efficient Leave-one-out Error Computation

We detail how to efficiently compute leave-one-out (LOO) errors for KNN models on sparse data, as done for our evaluation in Section 5. Algorithm 3 returns the leave-one-out errors ϕ for all n elements of the dataset $\mathcal{D}$ in a parallel loop over the validation set $\mathcal{D}_{val}$. For each validation sample $\mathbf{x}_q$, we retrieve its $k+1$ corresponding neighbors and compute the original utility v_q. Next, we

loop over the training samples forming the neighbors, maintain the pre-aggregate σ_q, compute the utility difference δ_{qi} when removing $\mathbf{x}_{\alpha_i}$ from the neighbor set, and update the corresponding LOO error accordingly. The time complexity of Algorithm 3 is $O(|\mathcal{D}_{val}|\, k)$ as it processes $k+1$ neighbors for each validation sample from $\mathcal{D}_{val}$.

Algorithm 3 Scalable leave-one-out error computation for the interaction data $\mathcal{D}$ with respect to a KNN model on sparse data.

function LOO($\mathcal{D}, V, k, \mathcal{D}_{val}$)
  $\phi = \{\phi_1, ..., \phi_n\} \leftarrow 0$ // Initialise LOO values
  parfor $\mathbf{x}_q \in \mathcal{D}_{val}$ // Iterate over validation samples
    $\mathbf{N}_q = \{\mathbf{x}_{\alpha_1}, \ldots, \mathbf{x}_{\alpha_{k+1}}\} \leftarrow f_{ret}(\mathbf{x}_q, \mathcal{D}, k+1)$ // Retrieve neighbors
    Initialize pre-aggregate σ_q from $\{\mathbf{x}_{\alpha_1}, \ldots, \mathbf{x}_{\alpha_k}\}$
    $v_q \leftarrow V(\mathbf{x}_q, f_{pred}(\sigma_q))$ // Original utility for validation sample
    for $i \in k \ldots 1$:
      Include $\mathbf{x}_{\alpha_{i+1}}$ into pre-aggregate σ_q
      Remove $\mathbf{x}_{\alpha_i}$ from pre-aggregate σ_q
      $\delta_{qi} \leftarrow v_q - V(\mathbf{x}_q, f_{pred}(\mathbf{x}_q, \sigma_q))$ // Change in utility
      $\phi_{\alpha_i} \leftarrow \phi_{\alpha_i} + \delta_{qi}$ // Update LOO for current neighbor

  return Leave-one-out errors $\phi = \{\phi_1, \ldots, \phi_n\}$

References

[1] Ziawasch Abedjan, Xu Chu, Dong Deng, Raul Castro Fernandez, Ihab F. Ilyas, Mourad Ouzzani, Paolo Papotti, Michael Stonebraker, and Nan Tang. 2016. Detecting Data Errors: Where are we and what needs to be done? *Proc. VLDB Endow.* 9, 12 (2016), 993–1004. doi:10.14778/2994509.2994518

[2] Mozhdeh Ariannezhad, Ming Li, Sebastian Schelter, and Maarten de Rijke. 2023. A Personalized Neighborhood-based Model for Within-basket Recommendation in Grocery Shopping. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining, WSDM 2023, Singapore, 27 February 2023 - 3 March 2023*, Tat-Seng Chua, Hady W. Lauw, Luo Si, Evimaria Terzi, and Panayiotis Tsaparas (Eds.). ACM, 87–95. doi:10.1145/3539597.3570417

[3] Ricardo Baeza-Yates and Berthier Ribeiro-Neto. 1999. *Modern information retrieval*. Vol. 463. ACM press New York.

[4] Eric Breck, Neoklis Polyzotis, Sudip Roy, Steven Whang, and Martin Zinkevich. 2019. Data Validation for Machine Learning. In *Proceedings of the Second Conference on Machine Learning and Systems, SysML 2019, Stanford, CA, USA, March 31 - April 2, 2019*, Ameet Talwalkar, Virginia Smith, and Matei Zaharia (Eds.). mlsys.org. https://proceedings.mlsys.org/paper_files/paper/2019/hash/928f1160e52192e3e0017fb63ab65391-Abstract.html

[5] Buzzfeed. 2019. "Amazon's Choice" Does Not Necessarily Mean A Product Is Good. https://www.buzzfeednews.com/article/nicolenguyen/amazons-choice-bad-products

[6] Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. 2019. Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches. In *Proceedings of the 13th ACM Conference on Recommender Systems, RecSys 2019, Copenhagen, Denmark, September 16-20, 2019*, Toine Bogers, Alan Said, Peter Brusilovsky, and Domonkos Tikk (Eds.). ACM, 101–109. doi:10.1145/3298689.3347058

[7] Jiale Deng, Yanyan Shen, Ziyuan Pei, Youmin Chen, and Linpeng Huang. 2025. Influence Guided Context Selection for Effective Retrieval-Augmented Generation. In *Advances in Neural Information Processing Systems*, D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen (Eds.), Vol. 38. Curran Associates, Inc., 29225–29252. https://proceedings.neurips.cc/paper_files/paper/2025/file/2a07497c94cd24b20faa3fd14d847037-Paper-Conference.pdf

[8] Dario Di Palma. 2023. Retrieval-augmented Recommender System: Enhancing Recommender Systems with Large Language Models. In *Proceedings of the 17th ACM Conference on Recommender Systems, RecSys 2023, Singapore, Singapore, September 18-22, 2023*, Jie Zhang, Li Chen, Shlomo Berkovsky, Min Zhang, Tommaso Di Noia, Justin Basilico, Luiz Pizzato, and Yang Song (Eds.). ACM, 1369–1373. doi:10.1145/3604915.3608889

[9] Guglielmo Faggioli, Mirko Polato, and Fabio Aiolli. 2020. Recency Aware Collaborative Filtering for Next Basket Recommendation. In *Proceedings of the 28th ACM Conference on User Modeling, Adaptation and Personalization, UMAP 2020, Genoa, Italy, July 12-18, 2020*, Tsvi Kuflik, Ilaria Torre, Robin Burke, and Cristina Gena (Eds.). ACM, 80–87. doi:10.1145/3340631.3394850

[10] Lampros Flokas, Weiyuan Wu, Yejia Liu, Jiannan Wang, Nakul Verma, and Eugene Wu. 2022. Complaint-Driven Training Data Debugging at Interactive Speeds. In *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Zachary G. Ives, Angela Bonifati, and Amr El Abbadi (Eds.). ACM, 369–383. doi:10.1145/3514221.3517849

[11] Diksha Garg, Priyanka Gupta, Pankaj Malhotra, Lovekesh Vig, and Gautam Shroff. 2019. Sequence and Time Aware Neighborhood for Session-based Recommendations: STAN. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2019, Paris, France, July 21-25, 2019*, Benjamin Piwowarski, Max Chevalier, Éric Gaussier, Yoelle Maarek, Jian-Yun Nie, and Falk Scholer (Eds.). ACM, 1069–1072. doi:10.1145/3331184.3331322

[12] Amirata Ghorbani and James Y. Zou. 2019. Data Shapley: Equitable Valuation of Data for Machine Learning. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA (Proceedings of Machine Learning Research, Vol. 97)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.). PMLR, 2242–2251. http://proceedings.mlr.press/v97/ghorbani19c.html

[13] Zayd Hammoudeh and Daniel Lowd. 2024. Training data influence analysis and estimation: a survey. *Mach. Learn.* 113, 5 (2024), 2351–2403. doi:10.1007/S10994-023-06495-7

[14] Alireza Heidari, Joshua McGrath, Ihab F. Ilyas, and Theodoros Rekatsinas. 2019. HoloDetect: Few-Shot Learning for Error Detection. In *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*, Peter Boncz, Stefan Manegold, Anastasia Ailamaki, Amol Deshpande, and Tim Kraska (Eds.). ACM, 829–846. doi:10.1145/3299869.3319888

[15] Balázs Hidasi and Alexandros Karatzoglou. 2018. Recurrent Neural Networks with Top-k Gains for Session-based Recommendations. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management, CIKM 2018, Torino, Italy, October 22-26, 2018*, Alfredo Cuzzocrea, James Allan, Norman W. Paton, Divesh Srivastava, Rakesh Agrawal, Andrei Z. Broder, Mohammed J. Zaki, K. Selçuk Candan, Alexandros Labrinidis, Assaf Schuster, and Haixun Wang (Eds.). ACM, 843–852. doi:10.1145/3269206.3271761

[16] Yupeng Hou, Binbin Hu, Zhiqiang Zhang, and Wayne Xin Zhao. 2022. CORE: Simple and Effective Session-based Recommendation within Consistent Representation Space. In *SIGIR '22: The 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, Madrid, Spain, July 11 - 15, 2022*, Enrique Amigó, Pablo Castells, Julio Gonzalo, Ben Carterette, J. Shane Culpepper, and Gabriella Kazai (Eds.). ACM, 1796–1801. doi:10.1145/3477495.3531955

[17] Haoji Hu, Xiangnan He, Jinyang Gao, and Zhi-Li Zhang. 2020. Modeling Personalized Item Frequency Information for Next-basket Recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval, SIGIR 2020, Virtual Event, China, July 25-30, 2020*, Jimmy X. Huang, Yi Chang, Xueqi Cheng, Jaap Kamps, Vanessa Murdock, Ji-Rong Wen, and Yiqun Liu (Eds.). ACM, 1071–1080. doi:10.1145/3397271.3401066

[18] Dietmar Jannach and Malte Ludewig. 2017. When Recurrent Neural Networks meet the Neighborhood for Session-Based Recommendation. In *Proceedings of the Eleventh ACM Conference on Recommender Systems, RecSys 2017, Como, Italy, August 27-31, 2017*, Paolo Cremonesi, Francesco Ricci, Shlomo Berkovsky, and Alexander Tuzhilin (Eds.). ACM, 306–310. doi:10.1145/3109859.3109872

[19] Ruoxi Jia, David Dao, Boxin Wang, Frances Ann Hubis, Nezihe Merve Gürel, Bo Li, Ce Zhang, Costas J. Spanos, and Dawn Song. 2019. Efficient Task-Specific Data Valuation for Nearest Neighbor Algorithms. *Proc. VLDB Endow.* 12, 11 (2019), 1610–1623. doi:10.14778/3342263.3342637

[20] Ruoxi Jia, Fan Wu, Xuehui Sun, Jiacen Xu, David Dao, Bhavya Kailkhura, Ce Zhang, Bo Li, and Dawn Song. 2021. Scalability vs. Utility: Do We Have To Sacrifice One for the Other in Data Importance Quantification?. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021, virtual, June 19-25, 2021*. Computer Vision Foundation / IEEE, 8239–8247. doi:10.1109/CVPR46437.2021.00814

[21] Bojan Karlaš, David Dao, Matteo Interlandi, Sebastian Schelter, Wentao Wu, and Ce Zhang. 2024. Data Debugging with Shapley Importance over Machine Learning Pipelines. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. https://openreview.net/forum?id=qxGXjWxabq

[22] Barrie Kersbergen and Sebastian Schelter. 2021. Learnings from a Retail Recommendation System on Billions of Interactions at bol.com. In *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*. IEEE, 2447–2452. doi:10.1109/ICDE51399.2021.00277

[23] Barrie Kersbergen, Olivier Sprangers, Bojan Karlaš, Maarten de Rijke, and Sebastian Schelter. 2025. Scalable Data Debugging for Neighborhood-based Recommendation with Data Shapley Values. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems, RecSys 2025, Prague, Czech Republic, September 22-26, 2025*, Mária Bieliková, Pavel Kordík, Markus Schedl, Marco de Gemmis, Sole Pera, Rodrigo Alves, Olivier Jeunen, and Vito Ostuni (Eds.). ACM, 441–450. doi:10.1145/3705328.3748049

[24] Barrie Kersbergen, Olivier Sprangers, and Sebastian Schelter. 2022. Serenade - Low-Latency Session-Based Recommendation in e-Commerce at Scale. In *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Zachary G. Ives, Angela Bonifati, and Amr El Abbadi (Eds.). ACM, 150–159. doi:10.1145/3514221.3517901

[25] Pang Wei Koh and Percy Liang. 2017. Understanding Black-box Predictions via Influence Functions. In *Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017 (Proceedings of Machine Learning Research, Vol. 70)*, Doina Precup and Yee Whye Teh (Eds.). PMLR, 1885–1894. http://proceedings.mlr.press/v70/koh17a.html

[26] Harold William Kuhn and Albert William Tucker. 1953. *Contributions to the Theory of Games.* Number 28. Princeton University Press.

[27] Yongchan Kwon and James Zou. 2022. Beta Shapley: a Unified and Noise-reduced Data Valuation Framework for Machine Learning. In *International Conference on Artificial Intelligence and Statistics, AISTATS 2022, 28-30 March 2022, Virtual Event (Proceedings of Machine Learning Research, Vol. 151)*, Gustau Camps-Valls, Francisco J. R. Ruiz, and Isabel Valera (Eds.). PMLR, 8780–8802. https://proceedings.mlr.press/v151/kwon22a.html

[28] Sara Latifi, Noemi Mauro, and Dietmar Jannach. 2021. Session-aware Recommendation: A Surprising Quest for the State-of-the-Art. *Inf. Sci.* 573 (2021), 291–315. doi:10.1016/J.INS.2021.05.048

[29] Ming Li, Sami Jullien, Mozhdeh Ariannezhad, and Maarten de Rijke. 2023. A Next Basket Recommendation Reality Check. *ACM Trans. Inf. Syst.* 41, 4 (2023), 116:1–116:29. doi:10.1145/3587153

[30] Peng Li, Xi Rao, Jennifer Blase, Yue Zhang, Xu Chu, and Ce Zhang. 2021. CleanML: A Study for Evaluating the Impact of Data Cleaning on ML Classification Tasks. In *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*. IEEE, 13–24. doi:10.1109/ICDE51399.2021.00009

[31] Greg Linden, Brent Smith, and Jeremy York. 2003. Amazon.com Recommendations: Item-to-Item Collaborative Filtering. *IEEE Internet Comput.* 7, 1 (2003), 76–80. doi:10.1109/MIC.2003.1167344

[32] Zifan Liu, Zhechun Zhou, and Theodoros Rekatsinas. 2022. Picket: Guarding against Corrupted Data in Tabular Data during Learning and Inference. *VLDB J.* 31, 5 (2022), 927–955. doi:10.1007/S00778-021-00699-W

[33] Malte Ludewig and Dietmar Jannach. 2018. Evaluation of Session-based Recommendation Algorithms. *User Model. User Adapt. Interact.* 28, 4-5 (2018), 331–390. doi:10.1007/S11257-018-9209-6

[34] Malte Ludewig, Noemi Mauro, Sara Latifi, and Dietmar Jannach. 2019. Performance Comparison of Neural and Non-neural Approaches to Session-based Recommendation. In *Proceedings of the 13th ACM Conference on Recommender Systems, RecSys 2019, Copenhagen, Denmark, September 16-20, 2019*, Toine Bogers, Alan Said, Peter Brusilovsky, and Domonkos Tikk (Eds.). ACM, 462–466. doi:10.1145/3298689.3347041

[35] Malte Ludewig, Noemi Mauro, Sara Latifi, and Dietmar Jannach. 2021. Empirical Analysis of Session-based Recommendation Algorithms. *User Model. User Adapt. Interact.* 31, 1 (2021), 149–181. doi:10.1007/S11257-020-09277-1

[36] Xuan Luo and Jian Pei. 2024. Applications and Computation of the Shapley Value in Databases and Machine Learning. In *Companion of the 2024 International Conference on Management of Data, SIGMOD/PODS 2024, Santiago, Chile, June 9-15, 2024*, Pablo Barceló, Nayat Sánchez-Pi, Alexandra Meliou, and S. Sudarshan (Eds.). ACM, 630–635. doi:10.1145/3626246.3654680

[37] Channel 4 News. 2017. Potentially Deadly Bomb Ingredients are 'Frequently Bought Together' on Amazon. https://www.channel4.com/news/potentially-deadly-bomb-ingredients-on-amazon

[38] Vice News. 2023. AI-Generated Books of Nonsense Are All Over Amazon's Bestseller Lists. https://www.vice.com/en/article/ai-generated-books-of-nonsense-are-all-over-amazons-bestseller-lists/

[39] Romila Pradhan, Jiongli Zhu, Boris Glavic, and Babak Salimi. 2022. Interpretable Data-Based Explanations for Fairness Debugging. In *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Zachary G. Ives, Angela Bonifati, and Amr El Abbadi (Eds.). ACM, 247–261. doi:10.1145/3514221.3517886

[40] Sergey Redyuk, Zoi Kaoudi, Volker Markl, and Sebastian Schelter. 2021. Automating Data Quality Validation for Dynamic Data Ingestion. In *Proceedings of the 24th International Conference on Extending Database Technology, EDBT 2021, Nicosia, Cyprus, March 23 - 26, 2021*, Yannis Velegrakis, Demetris Zeinalipour-Yazti, Panos K. Chrysanthis, and Francesco Guerra (Eds.). OpenProceedings.org, 61–72. doi:10.5441/002/EDBT.2021.07

[41] Paul Resnick, Neophytos Iacovou, Mitesh Suchak, Peter Bergstrom, and John Riedl. 1994. GroupLens: An Open Architecture for Collaborative Filtering of Netnews. In *CSCW '94, Proceedings of the Conference on Computer Supported Cooperative Work, Chapel Hill, NC, USA, October 22-26, 1994*, John B. Smith, F. Donelson Smith, and Thomas W. Malone (Eds.). ACM, 175–186. doi:10.1145/192844.192905

[42] Badrul Munir Sarwar, George Karypis, Joseph A. Konstan, and John Riedl. 2001. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the Tenth International World Wide Web Conference, WWW 10, Hong Kong, China, May 1-5, 2001*, Vincent Y. Shen, Nobuo Saito, Michael R. Lyu, and Mary Ellen Zurko (Eds.). ACM, 285–295. doi:10.1145/371920.372071

[43] Sebastian Schelter, Stefan Grafberger, Shubha Guha, Bojan Karlaš, and Ce Zhang. 2023. Proactively Screening Machine Learning Pipelines with ARGUSEYES. In *Companion of the 2023 International Conference on Management of Data, SIGMOD/PODS 2023, Seattle, WA, USA, June 18-23, 2023*, Sudipto Das, Ippokratis Pandis, K. Selçuk Candan, and Sihem Amer-Yahia (Eds.). ACM, 91–94. doi:10.1145/3555041.3589682

[44] Sebastian Schelter, Stefan Grafberger, Philipp Schmidt, Tammo Rukat, Mario Kießling, Andrey Taptunov, Felix Bießmann, and Dustin Lange. 2019. Differential Data Quality Verification on Partitioned Data. In *35th IEEE International Conference on Data Engineering, ICDE 2019, Macao, China, April 8-11, 2019*. IEEE, 1940–1945. doi:10.1109/ICDE.2019.00210

[45] Sebastian Schelter, Dustin Lange, Philipp Schmidt, Meltem Celikel, Felix Bießmann, and Andreas Grafberger. 2018. Automating Large-Scale Data Quality Verification. *Proc. VLDB Endow.* 11, 12 (2018), 1781–1794. doi:10.14778/3229863.3229867

[46] Shreya Shankar, Labib Fawaz, Karl Gyllstrom, and Aditya G. Parameswaran. 2023. Moving Fast With Broken Data. *CoRR* abs/2303.06094 (2023). arXiv:2303.06094 doi:10.48550/ARXIV.2303.06094

[47] Giuseppe Spillo, Allegra De Filippo, Cataldo Musto, Michela Milano, and Giovanni Semeraro. 2023. Towards Sustainability-aware Recommender Systems: Analyzing the Trade-off Between Algorithms Performance and Carbon Footprint. In *Proceedings of the 17th ACM Conference on Recommender Systems, RecSys 2023, Singapore, Singapore, September 18-22, 2023*, Jie Zhang, Li Chen, Shlomo Berkovsky, Min Zhang, Tommaso Di Noia, Justin Basilico, Luiz Pizzato, and Yang Song (Eds.). ACM, 856–862. doi:10.1145/3604915.3608840

[48] Qiaoyu Tan, Jianwei Zhang, Jiangchao Yao, Ninghao Liu, Jingren Zhou, Hongxia Yang, and Xia Hu. 2021. Sparse-Interest Network for Sequential Recommendation. In *WSDM '21, The Fourteenth ACM International Conference on Web Search and Data Mining, Virtual Event, Israel, March 8-12, 2021*, Liane Lewin-Eytan, David Carmel, Elad Yom-Tov, Eugene Agichtein, and Evgeniy Gabrilovich (Eds.). ACM, 598–606. doi:10.1145/3437963.3441811

[49] Ars Technica. 2024. Lazy Use of AI Leads to Amazon Products Called "I Cannot Fulfill that Request". https://arstechnica.com/ai/2024/01/lazy-use-of-ai-leads-to-amazon-products-called-i-cannot-fulfill-that-request/

[50] New York Times. 2022. Lawmakers Press Amazon on Sales of Chemical Used in Suicides. https://www.nytimes.com/2022/02/04/technology/amazon-suicide-poison-preservative.html

[51] Maria Vechtomova et al. 2023. Databricks AI Summit: Streamlining API Deploy ML Models Across Multiple Brands: Ahold Delhaize's Experience on Serverless. Retrieved July 5, 2024 from https://www.youtube.com/watch?v=GSJFyoBiCXk

[52] Tobias Vente, Lukas Wegmeth, Alan Said, and Joeran Beel. 2024. From Clicks to Carbon: The Environmental Toll of Recommender Systems. In *Proceedings of the 18th ACM Conference on Recommender Systems, RecSys 2024, Bari, Italy, October 14-18, 2024*, Tommaso Di Noia, Pasquale Lops, Thorsten Joachims, Katrien Verbert, Pablo Castells, Zhenhua Dong, and Ben London (Eds.). ACM, 580–590. doi:10.1145/3640457.3688074

[53] Alessandro Vicenti, Cataldo Musto, Giuseppe Spillo, Allegra De Filippo, Michela Milano, and Giovanni Semeraro. 2025. Estimating Product Carbon Footprint via Large Language Models for Sustainable Recommender Systems. In *Recommender Systems for Sustainability and Social Good - Second International Workshop, RecSoGood 2025, Prague, Czech Republic, September 26, 2025, Proceedings (Communications in Computer and Information Science, Vol. 2802)*, Ludovico Boratto, Allegra De Filippo, Elisabeth Lex, Noemi Mauro, Francesca Maridina Malloci, and Francesco Ricci (Eds.). Springer, 43–56. doi:10.1007/978-3-032-13342-7_4

[54] Jiachen T. Wang and Ruoxi Jia. 2023. Data Banzhaf: A Robust Data Valuation Framework for Machine Learning. In *International Conference on Artificial Intelligence and Statistics, 25-27 April 2023, Palau de Congressos, Valencia, Spain (Proceedings of Machine Learning Research, Vol. 206)*, Francisco J. R. Ruiz, Jennifer G. Dy, and Jan-Willem van de Meent (Eds.). PMLR, 6388–6421. https://proceedings.mlr.press/v206/wang23e.html

[55] Weiyuan Wu, Lampros Flokas, Eugene Wu, and Jiannan Wang. 2020. Complaint-driven Training Data Debugging for Query 2.0. In *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020*, David Maier, Rachel Pottinger, AnHai Doan, Wang-Chiew Tan, Abdussalam Alawini, and Hung Q. Ngo (Eds.). ACM, 1317–1334. doi:10.1145/3318464.3389696

[56] Zhouhang Xie, Junda Wu, Hyunsik Jeon, Zhankui He, Harald Steck, Rahul Jha, Dawen Liang, Nathan Kallus, and Julian J. McAuley. 2024. Neighborhood-Based Collaborative Filtering for Conversational Recommendation. In *Proceedings of the 18th ACM Conference on Recommender Systems, RecSys 2024, Bari, Italy, October 14-18, 2024*, Tommaso Di Noia, Pasquale Lops, Thorsten Joachims, Katrien Verbert, Pablo Castells, Zhenhua Dong, and Ben London (Eds.). ACM, 1045–1050. doi:10.1145/3640457.3688191

[57] Chengfeng Xu, Pengpeng Zhao, Yanchi Liu, Victor S. Sheng, Jiajie Xu, Fuzhen Zhuang, Junhua Fang, and Xiaofang Zhou. 2019. Graph Contextualized Self-Attention Network for Session-based Recommendation. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10-16, 2019*, Sarit Kraus (Ed.). ijcai.org, 3940–3946. doi:10.24963/IJCAI.2019/547

[58] Yansen Zhang, Xiaokun Zhang, Ziqiang Cui, and Chen Ma. 2025. Shapley Value-driven Data Pruning for Recommender Systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, V.2, KDD 2025,*

Toronto ON, Canada, August 3-7, 2025, Luiza Antonie, Jian Pei, Xiaohui Yu, Flavio Chierichetti, Hady W. Lauw, Yizhou Sun, and Srinivasan Parthasarathy (Eds.). ACM, 3879–3888. doi:10.1145/3711896.3737127

[59] Wayne Xin Zhao, Yupeng Hou, Xingyu Pan, Chen Yang, Zeyu Zhang, Zihan Lin, Jingsen Zhang, Shuqing Bian, Jiakai Tang, Wenqi Sun, Yushuo Chen, Lanling Xu, Gaowei Zhang, Zhen Tian, Changxin Tian, Shanlei Mu, Xinyan Fan, Xu Chen, and Ji-Rong Wen. 2022. RecBole 2.0: Towards a More Up-to-Date Recommendation Library. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management, Atlanta, GA, USA, October 17-21, 2022*, Mohammad Al Hasan and Li Xiong (Eds.). ACM, 4722–4726. doi:10.1145/3511808.3557680

[60] Yaochen Zhu, Chao Wan, Harald Steck, Dawen Liang, Yesu Feng, Nathan Kallus, and Jundong Li. 2025. Collaborative Retrieval for Large Language Model-based Conversational Recommender Systems. In *Proceedings of the ACM on Web Conference 2025, WWW 2025, Sydney, NSW, Australia, 28 April 2025- 2 May 2025*, Guodong Long, Michale Blumestein, Yi Chang, Liane Lewin-Eytan, Zi Helen Huang, and Elad Yom-Tov (Eds.). ACM, 3323–3334. doi:10.1145/3696410.3714908